

RF31 — Qualidade de Dados

Objetivo

Foi implementado um mecanismo de qualidade para validar os dados da camada Silver antes da publicação analítica na camada Gold.

O processo executa cinco testes distribuídos entre as dimensões:

- completude;
- validade;
- unicidade;
- consistência;
- integridade referencial.

Cada resultado é armazenado no PostgreSQL com:

- identificador da execução de qualidade;
- identificador da execução dos dados;
- fonte;
- teste;
- dimensão;
- fórmula;
- limite aceitável;
- severidade;
- ação;
- quantidade de registros;
- quantidade de registros inválidos;
- valor da métrica;
- status;
- data da execução.

Também é gerado um arquivo JSON para cada execução.

Estrutura de armazenamento

Foi criado o schema:

```
qualidade
```

Tabela de histórico:

```
qualidade.resultados
```

Cada linha representa o resultado de um teste de qualidade em uma determinada execução.

Também foi criada a view:

```
qualidade.vw\_evolucao\_metricas
```

Ela acompanha historicamente as métricas:

```
Q01\_COMPLETUDE
```

```
Q05\_INTEGRIDADE\_REFERENCIAL
```

Testes implementados

Q01 — Completude

Código:

```
Q01\_COMPLETUDE
```

Dimensão:

```
completude
```

Fonte:

```
silver.interacoes
```

Campos obrigatórios avaliados:

- usuario_id;
- conteudo_id;
- tipo_interacao;
- data_hora;
- tempo_consumido;
- percentual_conclusao;
- execucao_id;
- data_hora_padronizacao.

Fórmula:

```
(registros válidos / total de registros) \* 100
```

Limite aceitável:

100%

Severidade:

CRITICA

Ação em caso de falha:

Bloquear a publicação da Gold e revisar a etapa Silver/quarentena.

Q02 — Validade

Código:

`Q02\_VALIDADE\_PERCENTUAL`

Dimensão:

`validade`

Fonte:

`silver.interacoes`

Regra:

$$0 \leq \text{percentual_conclusao} \leq 100$$

Fórmula:

`(registros com percentual válido / total de registros) \* 100`

Limite aceitável:

`100%`

Severidade:

`ALTA`

Ação em caso de falha:

Registrar a ocorrência e revisar a regra de validação da Silver antes da próxima publicação

Q03 — Unicidade

Código:

Q03_UNICIDADE_CATALOGO

Dimensão:

unicidade

Fonte:

silver.catalogo

Regra:

conteudo_id deve ser único

Fórmula:

$(\text{conteudos únicos} / \text{total de registros}) \times 100$

Limite aceitável:

100%

Severidade:

CRITICA

Ação em caso de falha:

Bloquear a publicação da Gold e revisar a deduplicação do catálogo.

Q04 — Consistência

Código:

Q04_CONSISTENCIA_CONCLUSAO

Dimensão:

consistencia

Fonte:

silver.interacoes

Regra:

```
tipo\_interacao = "conclusão"

&#x20;           ↓

percentual\_conclusao = 100
```

Fórmula:

```
(registros consistentes / total de registros) \* 100
```

Limite aceitável:

```
100%
```

Severidade:

```
MEDIA
```

Ação em caso de falha:

```
Registrar ressalva e investigar interações de conclusão com percentual diferente de 100%
```

Uma falha nessa regra não bloqueia automaticamente a Gold, pois sua severidade foi definida como média.

Q05 — Integridade referencial

Código:

```
Q05\_INTEGRIDADE\_REFERENCIAL
```

Dimensão:

```
integridade\_referencial
```

Fontes:

```
silver.interacoes
```

```
&#x20;        ↓
```

```
silver.catalogo
```

Regra:

```
Todo conteudo\_id existente em silver.interacoes  
deve existir em silver.catalogo.
```

Fórmula:

```
(interações com conteúdo existente / total de interações) \* 100
```

Limite aceitável:

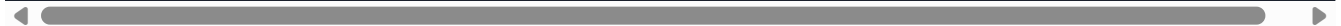
```
100%
```

Severidade:

```
CRITICA
```

Ação em caso de falha:

```
Bloquear a publicação da Gold e encaminhar registros órfãos para investigação/quarentena
```



Primeira execução — dados válidos

A primeira execução foi realizada sobre os dados existentes após o processamento da Silver.

Identificador:

```
19f9a23a-86be-4629-969c-2677ab2b28d3
```

Execução dos dados:

1

Resultados:

| Teste | Dimensão | Total | Inválidos | Métrica | Status |

|---|---|---:|---:|---:|---|

| Q01_COMPLETUDE | completude | 1001 | 0 | 100.000% | APROVADO |

| Q02_VALIDADE_PERCENTUAL | validade | 1001 | 0 | 100.000% | APROVADO |

| Q03_UNICIDADE_CATALOGO | unicidade | 1003 | 0 | 100.000% | APROVADO |

| Q04_CONSISTENCIA_CONCLUSAO | consistência | 1001 | 0 | 100.000% | APROVADO |

| Q05_INTEGRIDADE_REFERENCIAL | integridade referencial | 1001 | 0 | 100.000% | APROVADO |

Resultado geral:

SUCESSO

A publicação da Gold foi autorizada.

Simulação de falha crítica

Para validar o comportamento diante de uma falha real, foi inserido temporariamente um registro controlado em:

`silver.interacoes`

Identificação utilizada:

`execucao\_id = TESTE\_RF31\_FALHA`

O registro possuía:

```
usuario\_id = 999999  
conteudo\_id = 999999  
tipo\_interacao = NULL  
percentual\_conclusao = 50.00
```

A linha foi projetada para violar propositalmente duas regras críticas.

Violação de completude

```
tipo\_interacao = NULL
```

Viola:

```
Q01\_COMPLETUDE
```

Violação de integridade referencial

```
conteudo\_id = 999999
```

Esse conteúdo não existe em:

```
silver.catalogo
```

Viola:

```
Q05\_INTEGRIDADE\_REFERENCIAL
```

Execução com falha

Identificador da execução:

```
77efb690-e394-43ed-9179-2df45eb16fba
```

Execuções de dados identificadas:

```
1,TESTE\_RF31\_FALHA
```

Resultados críticos:

Teste	Total	Inválidos	Métrica	Status
-------	-------	-----------	---------	--------

--- ---	---	---	---	---
---------	-----	-----	-----	-----

Q01_COMPLETUDE	1002	1	99.900%	REPROVADO
----------------	------	---	---------	-----------

Q05_INTEGRIDADE_REFERENCIAL	1002	1	99.900%	REPROVADO
-----------------------------	------	---	---------	-----------

Resultado geral:

```
FALHA
```

Mensagem produzida:

```
PUBLICAÇÃO GOLD BLOQUEADA:
```

```
existe pelo menos uma regra crítica reprovada.
```

Código de saída do processo:

```
1
```

O código de saída diferente de zero permite que a etapa de qualidade interrompa as etapas dependentes durante a integração com o workflow de orquestração.

A integração desse mecanismo ao workflow Apache Hop será realizada na etapa de orquestração do RF22.

Recuperação

Após comprovar a falha, o registro artificial foi removido.

Quantidade de registros artificiais após a correção:

```
0
```

Quantidade original restaurada em:

```
silver.interacoes = 1001
```

Execução após recuperação

Identificador:

```
1b9e8ff7-6966-4471-9abc-48f96faa17f7
```

Execução dos dados:

```
1
```

Resultados:

Teste	Total	Inválidos	Métrica	Status
-------	-------	-----------	---------	--------

---	---	---	---	---
-----	-----	-----	-----	-----

Q01_COMPLETUDE	1001	0	100.000%	APROVADO
----------------	------	---	----------	----------

Q02_VALIDADE_PERCENTUAL	1001	0	100.000%	APROVADO
-------------------------	------	---	----------	----------

Q03_UNICIDADE_CATALOGO	1003	0	100.000%	APROVADO
------------------------	------	---	----------	----------

Q04_CONSISTENCIA_CONCLUSAO	1001	0	100.000%	APROVADO
----------------------------	------	---	----------	----------

Q05_INTEGRIDADE_REFERENCIAL	1001	0	100.000%	APROVADO
-----------------------------	------	---	----------	----------

Resultado geral:

SUCESSO

Código de saída:

0

Resultado:

Qualidade aprovada.

Publicação Gold autorizada.

Evolução das métricas

Foram escolhidas duas métricas para acompanhamento histórico:

Q01_COMPLETUDE

Q05_INTEGRIDADE_REFERENCIAL

A evolução observada foi:

| Execução | Q01 Completude | Q05 Integridade referencial | Situação |

|---|---|---|---|

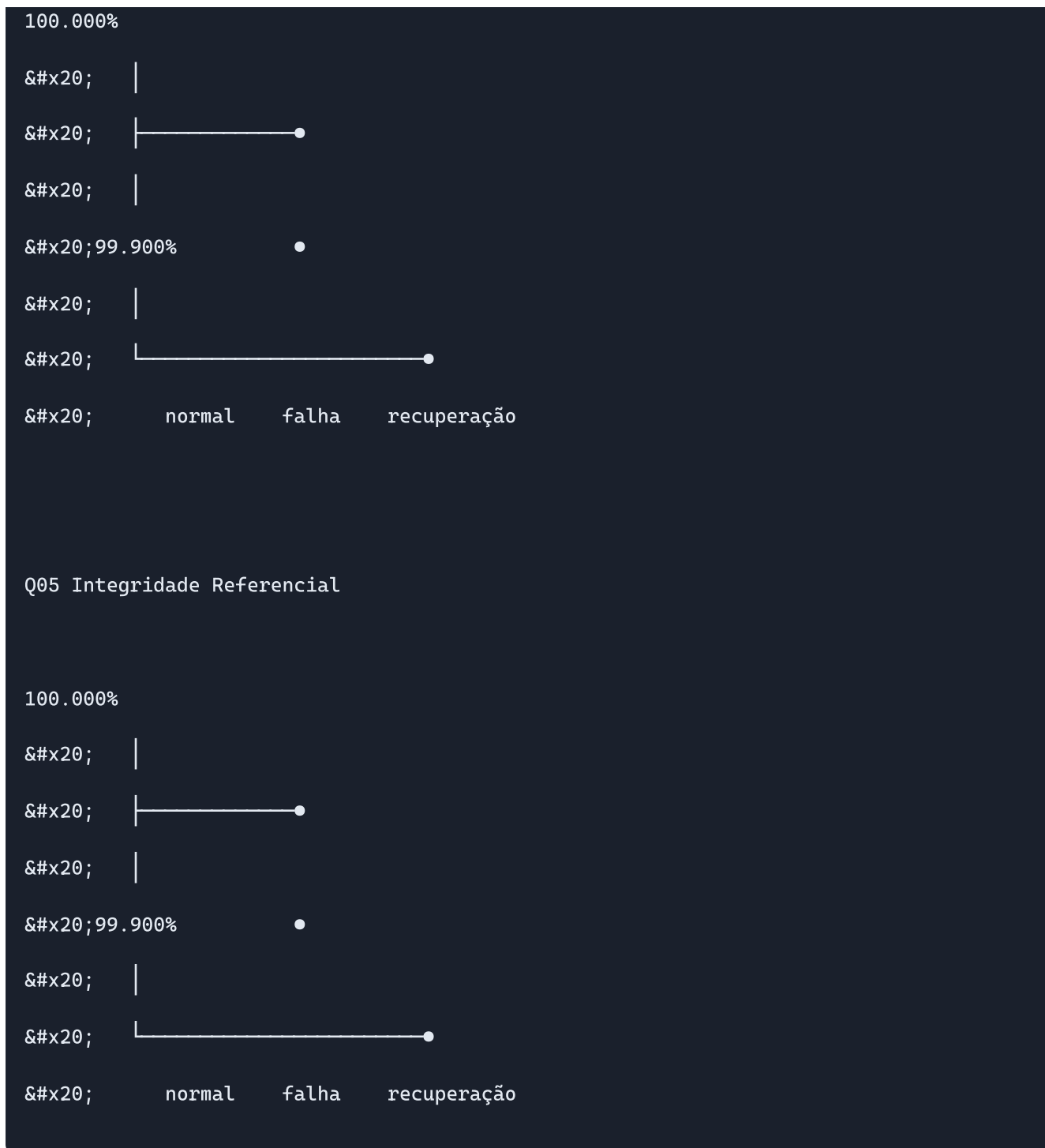
| Inicial | 100.000% | 100.000% | Dados válidos |

| Falha simulada | 99.900% | 99.900% | Falha crítica |

| Após recuperação | 100.000% | 100.000% | Dados recuperados |

Representação:

Q01 Completude



A evolução demonstra que o sistema consegue:

1. identificar dados válidos;
2. detectar degradação de qualidade;
3. bloquear processamento dependente diante de falha crítica;
4. registrar historicamente a ocorrência;
5. confirmar a recuperação após a correção.

Evidências JSON

Cada execução também gera um arquivo JSON em:

```
qualidade/resultados/
```

Arquivos produzidos:

```
qualidade\_19f9a23a-86be-4629-969c-2677ab2b28d3.json
```

```
qualidade\_77efb690-e394-43ed-9179-2df45eb16fba.json
```

```
qualidade\_1b9e8ff7-6966-4471-9abc-48f96faa17f7.json
```

Isso permite manter uma evidência independente do histórico armazenado no PostgreSQL.

Fluxo de decisão

O comportamento esperado para a orquestração é:

```
Silver
```

```
&#x20; |
```

```
&#x20; ▼
```

```
Testes de qualidade
```

```
&#x20; |
```

```
&#x20; |— sem falha crítica
```

```
&#x20; |      |
```

```
&#x20; |      ▼
```

```
&#x20; |      Beam
```

```

&#x20; |
&#x20; |   ▼
&#x20; |   Gold
&#x20; |
&#x20; | └─ falha crítica
&#x20; |
&#x20; |   ▼
&#x20; |   EXIT CODE 1
&#x20; |
&#x20; |   ▼
&#x20; |   Gold bloqueada

```

O executor já retorna:

```

0 = qualidade aprovada
1 = falha crítica

```

Esse comportamento será utilizado posteriormente pelo workflow Apache Hop.

Resultado

O RF31 foi implementado e validado com:

- cinco testes de qualidade;
- cinco dimensões diferentes;
- fórmula documentada por teste;
- limite aceitável por teste;
- severidade por teste;
- ação definida por teste;
- armazenamento por execução;

- armazenamento por fonte;
- evidências JSON;
- histórico no PostgreSQL;
- detecção de falha crítica;
- código de saída para bloqueio da Gold;
- recuperação após correção;
- evolução histórica de duas métricas.

O mecanismo de qualidade está pronto para integração ao workflow completo de orquestração.